

HACKATHON PROJECT

The next PDT model feature **is...**

Stacked orders quietly bend the promise we make to customers. Team **"PDTs are cool"** set out to find the features that explain it — and the markets where it matters most.

Meet the team ↓

1 THE TEAM

PDTs are cool

A cross-functional squad obsessed with one number customers actually feel: the **Promised / Predicted Delivery Time**. We came together for the hackathon to make that number honest for the hardest case — stacked orders.



Harnoor Chahal

Analysis & presentation



Zhamal Toktamysova

DTM simulations



Dongin Kim

DTM model & architecture



Asher Nehemiah

Offline analysis · Talabat



Grigoris Xenikakis

Presentation · e-food

2 THE PROBLEM

Why stacked orders?

Stacking is **~20% of global orders** and an important lever for profitability. Yet **high lateness** is one of the reasons — among others — that we don't stack to the highest potential.

Stacked orders break the PDT promise

When a rider carries two (or more) orders at once — a **stacked order** — the delivery the customer was promised competes with someone else's. The result: stacked orders arrive on time far less often than non-stacked ones. We measured the gap globally for Q1.

80.6%

On-time rate · **non-stacked** orders · 401.2M orders

70.1%

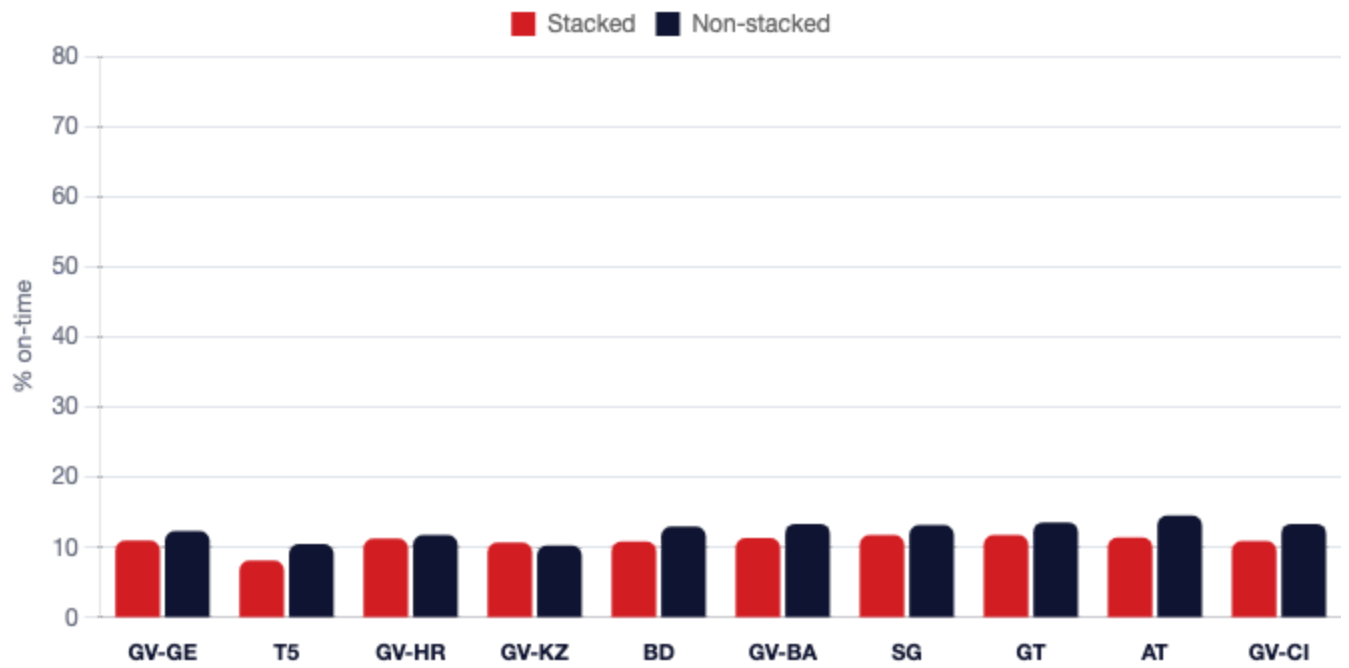
On-time rate · **stacked** orders · 155.0M orders

-10.4 pp

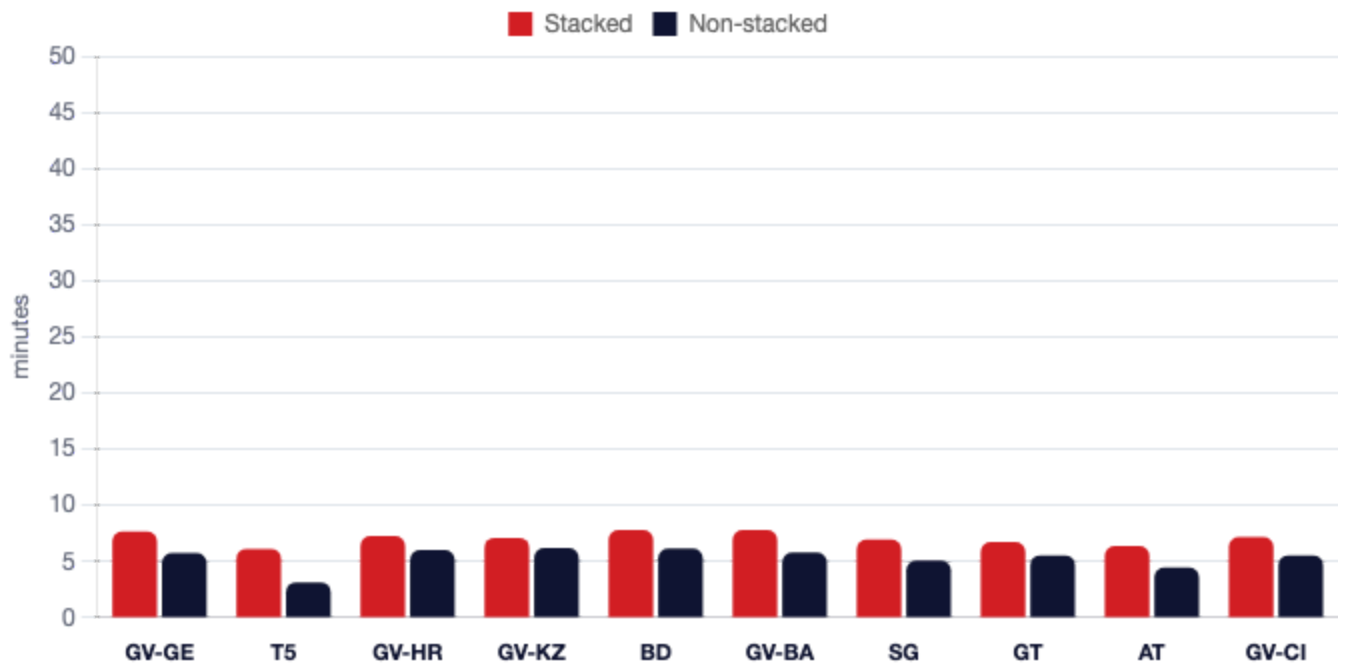
On-time **gap** — stacked orders are ~10pp less likely to be on time

Top 10 markets by the country-ranking dashboard's **default ranking** — stacked (**red**) vs non-stacked (**navy**). [see the live dashboard ↗](#)

% on-time (± 10 min)



Delivery time (min)



#	MARKET	ON-TIME STK	ON-TIME NON-STK	Δ	DT STK	DT NON-STK	Δ
1	GV-GE	59.2%	67.8%	-8.6	43.7m	33.4m	+10.3
2	T5	43.8%	57.4%	-13.6	35.7m	18.3m	+17.4
3	GV-HR	60.6%	64.6%	-4.0	42.2m	35.0m	+7.2
4	GV-KZ	58.9%	56.3%	+2.6	41.4m	36.0m	+5.4
5	BD	59.4%	71.1%	-11.7	45.3m	35.8m	+9.5
6	GV-BA	62.2%	73.4%	-11.2	45.5m	33.7m	+11.8
7	SG	64.7%	72.4%	-7.7	40.4m	29.5m	+10.9
8	GT	64.7%	74.6%	-9.9	39.1m	32.3m	+6.8
9	AT	62.5%	79.9%	-17.4	37.1m	26.0m	+11.1
10	GV-CI	59.8%	73.4%	-13.6	41.7m	32.3m	+9.4

Source: country-ranking dashboard data (cl.orders, trailing 30 days) — same numbers shown in the embedded dashboard's "Stacked vs non-stacked" tab. Δ = stacked – non-stacked; **red** = stacked performs worse. Global headline figures above are Q1 2026.

From model internals to the street

We worked top-down from the model, then bottom-up from operational reality — and met in the middle on the features that actually move stacked-order PDTs.

1 • Review the model structure

Sat down with our DTM owners **Dongin** and **Zhamal** to walk the current production architecture and the features used in its sub-models — and to find where stacked orders are (and aren't) represented.

2 • Break down the operations

Decomposed the problem into operational knowledge, focusing hardest on **dispatching** and **rider issues** — the two levers most tied to stacking.

3 • Talk to our rider friends

Filled up our knowledge gaps by talking to our friends in **rider operations** — ground-truthing our hypotheses against what actually happens on a stacked trip: acceptance, wait, and hand-off frictions.

The feature long-list

Whiteboarding the delivery journey end-to-end, we mapped candidate features stage by stage — then zeroed in on the dispatch/rider levers (and ultimately **Δ CPU**).

Vendor & prep (EPT)

Estimated prep time

Vendor restrictions

Order locale at vendor

Stacking rate / zone density at vendor

Pickup

Building access / "walls"

Foot traffic at vendor

Map & routing (Google) signals

Driving

Traffic on the line

Distance to customer

Speed of vehicle / vehicle type

Special errands & detours

At customer

Near-customer time

Walk-in time

Walk-out time at customer

Dispatch & rider ★ focus

Dispatch time

Rider acceptance time

Rider acceptance rate = $f(\text{accept time})$

Mean delay per zone

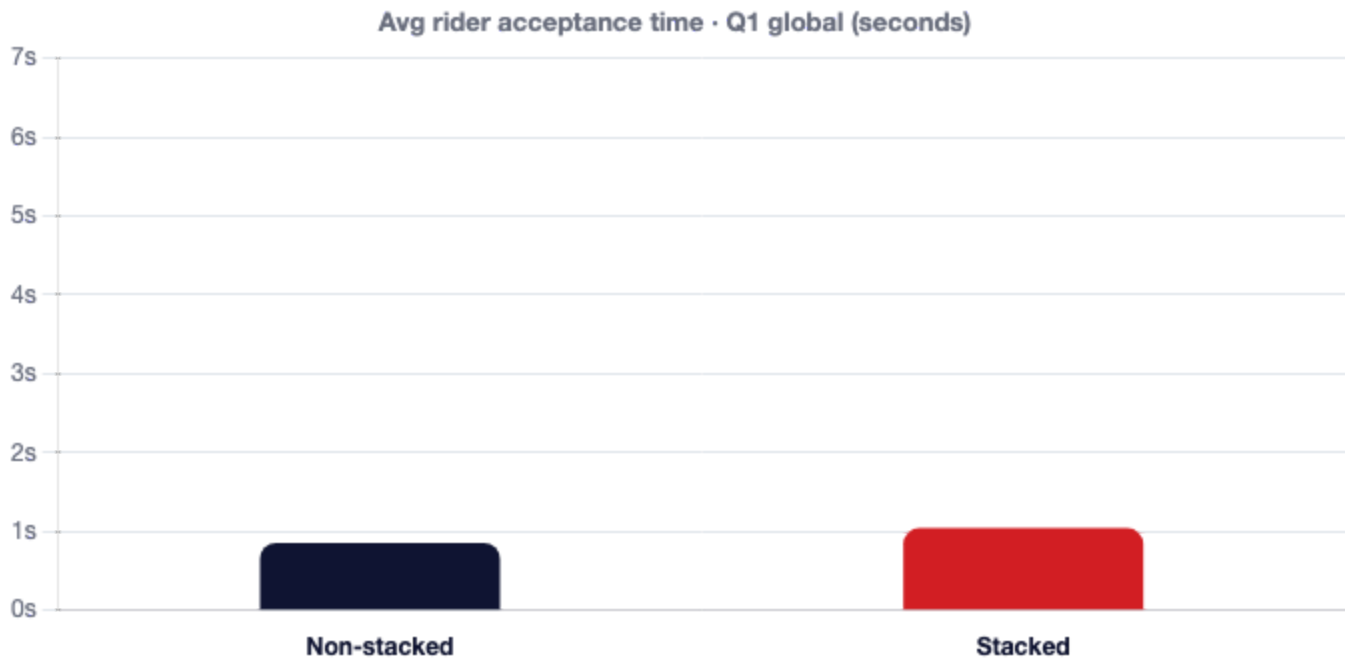
On-break / available riders

The pick: Δ CPU ★ tested

Gap between two orders' committed pickup times — the same signal the dispatcher already uses to gate stacking, but invisible to today's PDT model. (See §4.)

Rider acceptance time & the Δ CPU penalty

Two things pointed us at the dispatch layer. First, riders take measurably **longer to accept stacked orders**. Second, the dispatcher already gates stacking on **Δ CPU** — a signal today's PDT model never sees.



Source: `cl.orders · rider_reaction_time`, Q1 2026 global, completed non-pre-order orders. Stacked orders take **+1.2s (+22%)** longer to accept on average.

What is the Δ CPU penalty?

CPU = Committed Pick-Up time — when a rider is committed to collect an order. **Δ CPU** is the gap between the committed pickup times of two orders the algorithm is considering stacking together.

Dispatch only stacks them when **Δ CPU $\leq$ a max threshold (~4 min)**; beyond that, a penalty blocks the stack so a rider isn't sent to wait too long at the second vendor.

Example. Order 1 committed PU 10:15, Order 2 10:20 $\rightarrow \Delta$ CPU = 5 min $\rightarrow$ **no stack**. Once Order 1 is at vendor (10:16), Δ CPU = 4 min $\rightarrow$ **stack allowed**.

Source: LOGRO Squad Update — *MaxDeltaCPU* dispatch constraint. The dispatcher acts on Δ CPU; the PDT model doesn't — which is the gap we set out to close.

5 SELECTING MARKETS

Where to act first

To pick where stacked-order PDT fixes would pay off most, we built a live **country ranking dashboard** — a weighted rank-sum over KPI signals you can toggle in the browser. It re-ranks instantly, no re-query needed. The shortlist it produced: **gv-hr, gv-ci, gv-bg, gv-me, gv-ba, gt**.

Country Ranking Dashboard

Open full dashboard ↗

Interactive dashboard — view live at predict-timing-unveiled.ngrok-free.dev/country_ranking_dashboard.html

We put it in code

With the features and target markets in hand, we trained the DTM with the new Δ CPU feature against a control, on a dedicated `cp1-hackathon` branch, and ran an offline evaluation.

Inputs

New Δ CPU feature vs. control model. Target markets gv-hr, gv-ci, gv-bg, gv-me, gv-ba, gt.

Method

Trained on Metaflow — 1-month training window, execution date 2026-05-20 — and compared against the control models offline.

Output

Change in stacked vs. non-stacked PDT accuracy / on-time once the model accounts for Δ CPU.

How AI was used

Branch per feature

Each candidate feature (e.g. rider-acceptance) sat on its own git branch, so we could build, train, and evaluate features **independently and in parallel**, without conflicts.

A DTM-aware agent

On each branch we drove an AI agent wired into our DTM repo with **full company context** — it knows our model structure, our BigQuery schemas, and the team's conventions. So it didn't write generic code; it produced feature SQL and pipeline wiring that dropped straight into our existing DTM pipeline.

Days → hours

Describe the idea → the agent writes the BigQuery feature, wires it into the pipeline, and runs the per-country accuracy eval **the same way we already do it**.

Summary

- **Branch per feature** → parallel, isolated feature dev.
- **DTM-aware AI agent** (knows our model, BigQuery schema, conventions) → features built in hours, not days.
- **BigQuery end-to-end** → the agent builds the feature data and runs per-country evaluation directly.

"DTM-aware" = we set up CLAUDE.md and AI-context docs directly in the repo, so the agent reads our model, data, and conventions as built-in context.

Δ CPU cuts the stacked-order late tail

5 of 6 markets in. Of the two features we tested in parallel, **Δ CPU moved the needle** and rider-acceptance didn't — pointing to a real GMV opportunity.

4 / 5

Δ CPU — stacked markets with a shorter late tail (up to +5.0pp). On-time ~flat.

no lift

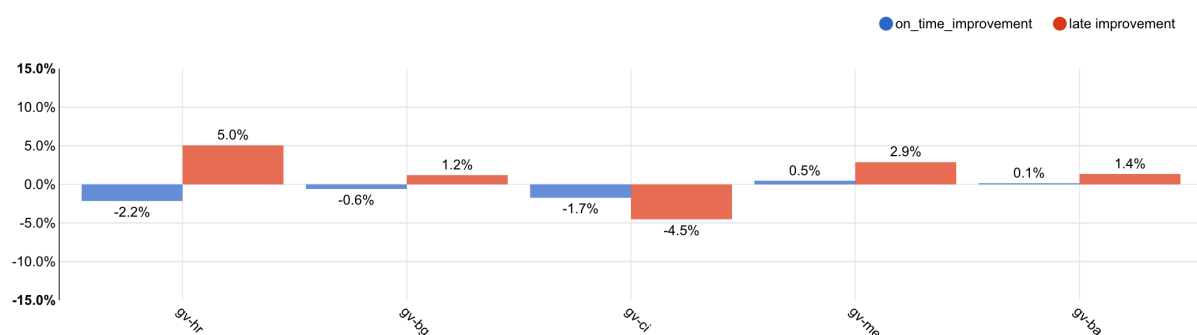
Rider acceptance — deltas within noise; dropped. Parallel testing paid off.

≈ €5–16 Mio

est. **annual GMV uplift** from fewer late-driven cancellations (central ~€14 Mio).

Δ CPU on stacked orders — on-time / late improvement (vs control, 5 markets)

DTM OnTime/Late Improvement



Positive = better (on-time up / lateness down). Late tail improves in 4/5 markets (gv-ci the exception); on-time broadly flat. Source: Metaflow cpl-hackathon, 1-month training, exec 2026-05-20 — only **gt** still running.

▼ All orders — context

▼ Second feature tested — rider acceptance (no lift)

▼ GMV estimate — how we get there

★ AT A GLANCE

The one-minute version

PROBLEM

Stacked orders (~20% of global orders) are **~10pp less likely** to be on time — 70% vs 81% (Q1, global).

APPROACH

Mapped the DTM, focused on **dispatch & rider** levers, talked to riders — then tested features in parallel with a **DTM-aware AI agent**.

RESULT

Δ CPU shortens the stacked-order **late tail** in 4 of 5 markets (up to +5pp). Rider-acceptance gave no lift.

IMPACT

≈ **€5–16 Mio/yr** GMV from fewer late-driven cancellations (central ~€14 Mio).

PDTs are cool,

but our team is COOLER 😎

PDTs are cool · Stacked Order PDT Hackathon · Customer Product Line Connect